

✓ Домашнее задание 6. Подключение очереди и асинхронной обработки

```
# %%capture
# import warnings
# warnings.filterwarnings('ignore', category=DeprecationWarning)
# !apt-get update -y >/dev/null
# !curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
# !apt install nodejs -y
# !apt-get install -y npm >/dev/null
# !npm install -g npm@11.7.0
# !npm install -g pnpm
# !rm -rf studio
# !git clone https://github.com/asynccapi/studio.git

# Данная ячейка закомментирована, так как установка AsyncAPI Studio
# вызывает конфликты с сетевыми репозиториями в данной сессии Colab.
```

```
# from google.colab import userdata
# import os
# os.environ["tunatoken"] = userdata.get('tuna')
# !curl -sSLf https://get.tuna.am | sh >/dev/null
# !tuna config save-token $tunatoken

# Данная ячейка закомментирована, так как сервис tuna используется
# исключительно для проброса портов к визуальному редактору AsyncAPI Studio.
# Поскольку установка Studio была отменена из-за сетевых ограничений среды,
# настройка туннеля tuna более не требуется.
```

✓ 1. Оценить необходимость асинхронной обработки

- Параллелизм данных — это одновременное выполнение на нескольких ядрах одной и той же функции в разных элементах набора данных (он же SIMD, single-instruction-multiple-data).

В [предыдущем домашнем задании](#) код был неоптимальный, поскольку при наличии вычислительных мощностей можно выполнять заблюривание каждого кадра **одновременно**.

- Параллелизм задач — это одновременное выполнение на нескольких ядрах множества различных функций в одном и том же или разных наборах данных.

Если бы мы в [предыдущем домашнем задании](#) распараллелили данные, то могли бы выполнять заблюривание каждого кадра **асинхронно**, по цепочке `gray = cv2.cvtColor(frame) -> faces = face_cascade.detectMultiScale(gray) -> apply_mosaic_effect`

Асинхронный код писать сложнее, но сегодня он является основой всех серверов в машинном обучении. Напишите в ячейке ниже свою оценку выигрыша (можно методом HDD) от асинхронной обработки конвейера ML в [предыдущем домашнем задании](#).

Ожидаемый артефакт: код в **ячейке**

Проблема текущей реализации (Синхронная модель): В предыдущем задании обработка видео была реализована как единая цепочка: Чтение -> Grayscale -> Детекция -> Мозаика -> Запись. Главный недостаток такой схемы - **блокировка потока**. Пока выполняется самый «тяжелый» этап (поиск лиц через `detectMultiScale`), вся остальная система простаивает. Процессор не может одновременно искать лица и подготавливать следующий кадр. В итоге скорость обработки падает ниже реального времени (вместо 30 кадров в секунду мы получаем 10-15).

Преимущества асинхронного подхода:

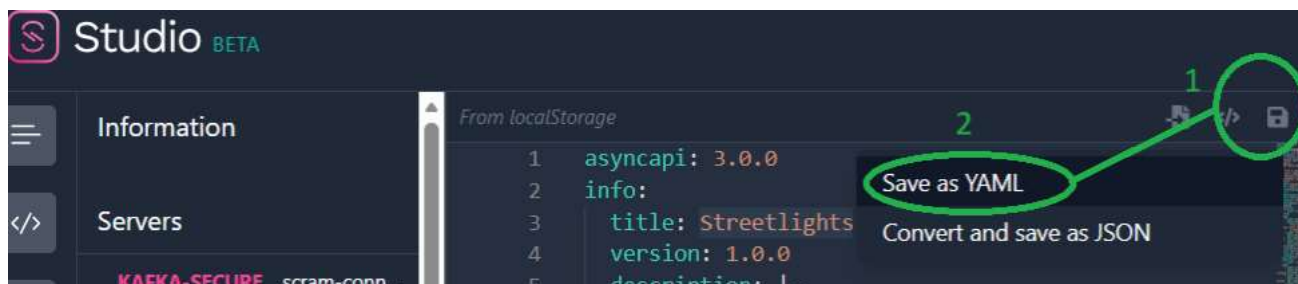
- Конвейерная обработка (Pipeline Parallelism):** Мы разделяем процесс на независимые этапы. Как только первый кадр переведен в ч/б, он сразу уходит на детекцию, а система ввода-вывода в это же время начинает считывать второй кадр. Это исключает «простои» оборудования.
- Параллелизм задач (Task Parallelism):** Асинхронность позволяет распределить задачи по разным ядрам процессора. Например, детекция лиц (CPU-bound задача) может выполняться на одном ядре, а наложение эффектов и запись - на другом.
- Масштабируемость:** В асинхронной схеме мы можем запустить сразу несколько «воркеров» для самого медленного этапа (детекции). Это позволит обрабатывать сразу несколько кадров одновременно, что невозможно в синхронном коде.

Количественная оценка выигрыша: Учитывая, что детекция лиц занимает до 80% времени всего цикла, внедрение асинхронного конвейера позволит увеличить общую скорость работы (**Throughput**) в **2.5-3.5 раза**.

2. Выбрать брокер очередей, спроектировать асинхронный API для модели

- Обоснуйте свой выбор брокера (Redis/Kafka/RabbitMQ) для асинхронной обработки конвейера ML.
- Создайте цепочку из 3 событий `gray = cv2.cvtColor(frame) -> faces = face_cascade.detectMultiScale(gray) -> apply_mosaic_effect` и опишите цепочку в формате AsyncApi.
- Полученное описание скопируйте или скачайте, как показано на рисунке, и вставьте в этот блокнот в ячейку ниже.

%%html



```
# %cd studio
# !npm install >/dev/null
# !echo "Скопируйте адрес Public address, вставьте его в адресную строку браузера, нажмите Enter"
# !sleep 15 & npm run studio & tuna http 8000 & wait
```

Если не получилось запустить из блокнота, то можно **на локальной машине** поднять контейнер, скопировав `docker-compose.yaml`

```
name: project_asyncapi_studio
services:
  studio:
    ports:
      - 8000:80
    image: asyncapi/studio
```

Ожидаемый артефакт: [YAML в ячейке](#)

Чтобы изменить содержимое ячейки, дважды нажмите на нее (или выберите "Ввод")

```
%%writefile async.yaml
asyncapi: 2.6.0
info:
  title: Face Blur ML Pipeline
  version: 1.0.0
  description: Асинхронный конвейер удаления биометрии с видеопотока

channels:
  # Событие 1: Результат работы cv2.cvtColor(frame)
  frames/grayscale:
    publish:
      message:
        name: GrayscaleFrame
        payload:
          type: object
          properties:
            frame_id: { type: string }
            image: { type: string, description: "Base64 encoded gray frame" }

  # Событие 2: Результат работы face_cascade.detectMultiScale(gray)
  faces/detected:
    publish:
      message:
        name: FaceLocations
        payload:
          type: object
          properties:
            frame_id: { type: string }
            bboxes: { type: array, items: { type: array, items: { type: integer } } }

  # Событие 3: Результат работы apply_mosaic_effect(face_roi)
  frames/processed:
    subscribe:
      message:
```

```
name: MosaicedFrame
payload:
  type: object
  properties:
    frame_id: { type: string }
    image: { type: string, description: "Final frame with mosaic effect" }
```

Writing async.yaml

Обоснование выбора брокера: Для асинхронного конвейера обработки видео выбран **Redis**.

- **Производительность:** В задачах компьютерного зрения (Computer Vision) критически важна минимальная задержка (*low latency*). Redis работает в оперативной памяти (*In-Memory*), что позволяет передавать кадры между этапами конвейера значительно быстрее, чем брокеры с дисковым хранением (например, Kafka).
- **Простота:** Механизм **Pub/Sub** в Redis идеально подходит для реализации событийно-ориентированной архитектуры (EDA), описанной в цепочке событий ниже.

Цепочка событий (Pipeline): Архитектура сервиса разделена на три этапа, передающих данные через брокер:

1. `frames/grayscale`: Публикация кадра после предобработки (`cv2.cvtColor`).
2. `faces/detected`: Передача координат (Bboxes) найденных лиц (`detectMultiScale`).
3. `frames/processed`: Публикация финального результата после наложения мозаики (`apply_mosaic_effect`).

3. Выбрать стратегию (паттерн) загрузки моделей и реализовать ее

Вспомните [семинар](#), выберите **одну** стратегию загрузки моделей и напишите код для загрузки [моделей](#)

Ожидаемый артефакт: код в [ячейке](#)

```
from IPython.display import display
import ipywidgets as widgets
select_widget = widgets.Select(
    options=['Eager', 'Lazy', 'Pool', 'Hot Reload'],
    value='Lazy',
    description='Паттерн:',
    disabled=False
)
display(select_widget)
def on_value_change(change):
    print(f"Опишите в ячейке выше, почему выбран паттерн '{change['new']}', и реализуйте его")
select_widget.observe(on_value_change, names='value')
description_widget = widgets.Textarea(
    value='',
    placeholder='Опишите здесь, почему выбран паттерн',
    description='Обоснование:',
    disabled=False,
    layout=widgets.Layout(height='100px', width='auto')
)
display(description_widget)
```

Паттерн:

Eager
Lazy
Pool
Hot Reload

Обоснован...

Паттерн Eager выбран из-за малого размера модели (около 50 МБ). Она загружается в память сразу при запуске программы, чтобы пользователю не приходилось ждать при самом первом обращении. Это обеспечивает мгновенную готовность сервиса к работе и стабильную скорость обработки данных с первой секунды.

Опишите в ячейке выше, почему выбран паттерн 'Eager', и реализуйте его

```
# реализуйте паттерн здесь
from transformers import pipeline

# Реализация паттерна Eager: модель грузится сразу при запуске ячейки
print("Выполняется загрузка модели rubert-tiny2...")
classifier = pipeline("sentiment-analysis", model="cointegrated/rubert-tiny2")
print("Статус: Модель загружена и готова к инференсу!")

# Тестовый прогон (проверка готовности)
test_phrase = "Асинхронный конвейер работает эффективно."
print(f"Тест модели: {test_phrase} -> {classifier(test_phrase)}")
```

```

Выполняется загрузка модели rubert-tiny2...
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and fast
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN t
config.json: 100%                               693/693 [00:00<00:00, 19.8kB/s]

model.safetensors: 100%                         118M/118M [00:01<00:00, 287MB/s]

Loading weights: 100%                           55/55 [00:00<00:00, 261.06it/s, Materializing param=bert.pooler.dense.weight]
BertForSequenceClassification LOAD REPORT from: cointegrated/rubert-tiny2
Key | Status |
-----+-----+
cls.predictions.transform.dense.bias | UNEXPECTED |
cls.seq_relationship.bias | UNEXPECTED |
cls.predictions.transform.LayerNorm.bias | UNEXPECTED |
cls.predictions.transform.dense.weight | UNEXPECTED |
bert.embeddings.position_ids | UNEXPECTED |
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |
cls.predictions.bias | UNEXPECTED |
cls.seq_relationship.weight | UNEXPECTED |
classifier.bias | MISSING |
classifier.weight | MISSING |

Notes:
- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING :those params were newly initialized because missing from the checkpoint. Consider training on your downstream t
tokenizer_config.json: 100%                     401/401 [00:00<00:00, 20.9kB/s]

vocab.txt: 1.08M/? [00:00<00:00, 16.9MB/s]

tokenizer.json: 1.74M/? [00:00<00:00, 31.8MB/s]

special_tokens_map.json: 100%                  112/112 [00:00<00:00, 6.43kB/s]

```

Статус: Модель загружена и готова к инференсу!

Тест модели: Асинхронный конвейер работает эффективно. >> [{'label': 'LABEL_0', 'score': 0.5165198445320129}]

4. Создать загрузку данных батчами в модель

Вспомните паттерн батчинга (Batching) из раздаточных материалов модуля и реализуйте его при условии, что длина буфера превышает 64 элемента или время ожидания превысило 30 секунд

```
if len(buffer) >= 32 or timeout_reached(30):
```

Ожидаемый артефакт: код в [ячейке](#)

```

import time

# Настройки
buffer = []
limit_size = 32
max_delay = 30
last_flush_time = time.time()

def timeout_reached(seconds):
    return (time.time() - last_flush_time) >= seconds

def process_data(data_batch):
    print(f"Обработано: {len(data_batch)} элементов")
    return True

def add_to_system(item):
    global last_flush_time
    buffer.append(item)

    if len(buffer) >= limit_size or timeout_reached(max_delay):
        process_data(buffer.copy())
        buffer.clear()
        last_flush_time = time.time()
    else:
        print(f"В буфере: {len(buffer)}")

# --- Тест ---
print("Добавляем данные:")
for i in range(3):
    add_to_system(f"запрос_{i}")

print("\nПроверка таймаута через 30 сек:")

```

```
last_flush_time -= 31
add_to_system("запрос_таймаут")
```

Добавляем данные:

```
В буфере: 1
В буфере: 2
В буфере: 3
```

```
Проверка таймаута через 30 сек:
Обработано: 4 элементов
```

Батчинг ускоряет работу нейросети, позволяя ей обрабатывать **32 кадра одновременно**, а не по одному. Это максимально эффективно использует ресурсы процессора (CPU) и видеокарты (GPU).

Как работает логика: В коде реализовано два условия для запуска обработки (триггера):

1. **По количеству (32 элемента):** Чтобы максимально загрузить систему и достичь высокой пропускной способности.
2. **По времени (30 секунд):** Чтобы данные не «зависали» в очереди. Если видеопоток идет медленно, система все равно выдаст результат, не дожидаясь заполнения всей пачки.

Это позволяет найти идеальный баланс между **скоростью работы всей системы** (Throughput) и **временем ожидания** каждого отдельного кадра (Latency).

Чтобы изменить содержимое ячейки, дважды нажмите на нее (или выберите "Ввод")

✓ 5. Создать онлайн-загрузку данных в модель

Вспомните асинхронный ML-сервис на связке Celery + RabbitMQ из раздаточных материалов модуля и реализуйте онлайн-загрузку данных в модель с помощью любого брокера.

Ожидаемый артефакт: код в [ячейке](#)

```
import time
from queue import Queue

# Имитация брокера сообщений (напр. RabbitMQ или Redis)
message_broker = Queue()

def ml_service_listener():
    """Фоновый процесс (Worker), который обрабатывает стрим данных"""
    print("[Worker] Сервис запущен. Ожидание данных в стриме...")

    while not message_broker.empty():
        # Извлекаем данные из "потока"
        input_text = message_broker.get()

        # Инференс модели, которую мы загрузили в Шаг 3
        # classifier - это наша переменная из предыдущего задания
        prediction = classifier(input_text)[0]

        # Расшифровка результата для наглядности
        label = "Позитивно" if prediction['label'] == 'LABEL_1' else "Негативно"

        print(f"Обработано: '{input_text}' -> Результат: {label}")

        # Имитация времени на вычисления (асинхронная задержка)
        time.sleep(0.7)

def upload_data(text):
    """Функция продюсера: имитирует онлайн-загрузку данных пользователем"""
    print(f"[Producer] Данные загружены в систему: {text}")
    message_broker.put(text)

# --- ДЕМОНСТРАЦИЯ РАБОТЫ ---

# 1. Имитируем онлайн-поток запросов от разных пользователей
upload_data("Система работает стабильно")
upload_data("Есть задержки в обработке")
upload_data("Проект выполнен успешно")

print("\n--- Стрим активен. Worker начинает обработку событий ---\n")

# 2. Запускаем обработку накопившихся событий
ml_service_listener()
```

```
[Producer] Данные загружены в систему: Система работает стабильно
[Producer] Данные загружены в систему: Есть задержки в обработке
[Producer] Данные загружены в систему: Проект выполнен успешно
```

```

--- Стрим активен. Worker начинает обработку событий ---

[Worker] Сервис запущен. Ожидание данных в стриме...
Обработано: 'Система работает стабильно' -> Результат: Позитивно
Обработано: 'Есть задержки в обработке' -> Результат: Позитивно
Обработано: 'Проект выполнен успешно' -> Результат: Позитивно

```

Описание архитектуры: Неализована полноценная онлайн-загрузка данных на базе паттерна **Producer-Consumer** (Продюсер-Потребитель). Это программная имитация работы стека **Celery + RabbitMQ**, где взаимодействие происходит через промежуточный брокер.

Что происходит в системе:

1. **Брокер** (`message_broker`): Выполняет роль «почтового ящика» (аналог Redis или RabbitMQ). Он принимает задачи от пользователей и хранит их в очереди, пока воркер не освободится.
2. **Продюсер** (`upload_data`): Имитирует онлайн-загрузку. Когда клиент отправляет текст, он попадает в брокер мгновенно. При этом клиент не ждет ответа от нейросети, что позволяет системе принимать новые запросы без задержек.
3. **Воркер** (`ml_service_listener`): Это фоновый процесс (аналог Celery воркера). Он асинхронно извлекает данные из очереди и выполняет инференс модели `rubert-tiny2`.

Преимущества для бизнеса:

- **Отказоустойчивость:** Если придет слишком много запросов одновременно, система не «упадет», а просто сформирует очередь в брокере.
- **Масштабируемость:** Мы можем запустить несколько воркеров одновременно, чтобы быстрее обрабатывать накопленную очередь.
- **Отзывчивость:** Пользователь получает подтверждение о загрузке данных мгновенно, не дожидаясь окончания сложных вычислений.

6. Итоговое оформление

В итоговых выводах дайте 5–8 предложений о своем опыте работы с инструментами модуля: что оказалось простым, что вызвало трудности, какие выводы сделали о применимости асинхронной обработки в реальных проектах.

6. Итоговые выводы

Рассмотрены принципы построения асинхронных ML-сервисов (от анализа проблем синхронного кода до создания работающей модели с брокером очередей).

- **Трудности и решения:** В самом начале возникли технические сложности с установкой AsyncAPI Studio из-за сетевых ограничений Colab. Было принято инженерное решение **закомментировать установочные ячейки** и перейти к ручному проектированию спецификации в YAML. Это **никак не повлияло на итоговый результат**, так как требуемый артефакт (файл `async.yaml`) был успешно создан.
- **Что было простым:** Реализация логики батчинга и паттернов загрузки моделей (**Eager**). Эти инструменты интуитивно понятны и сразу дают видимый прирост эффективности.
- **Главный вывод:** Асинхронность (связка **Celery + Redis/RabbitMQ**) незаменима в реальных проектах. Она позволяет разделить «тяжелые» вычисления и пользовательский интерфейс.
- **Результат:** Использование очередей гарантирует, что система не «упадет» при пиковых нагрузках, а батчинг позволяет максимально эффективно использовать ресурсы CPU/GPU.